



COMP 4300

Intro to Game Programming

Lecture 6

A2 Engine Architecture

Assignment 2 Explained (in-class)

Engine Architecture Progression

- Assignment 1
 - Getting used to C++ / API
 - All functionality in main.cpp
- Assignment 2
 - Separation of functionality into Classes
 - Wrote our own Vec2 class for 2D game math
 - Appearance of ECS classes
 - EntityManager handles all Entity data
 - ECS Systems functionality in Game.h functions

A2 Engine Architecture

GameEngine

- > Systems
- > EntityManager
 - > Entity
 - > Component

Components

- Pure Data
- Stored in Entity
- Each Component is its own class

CTransform

```
+ pos:      Vec2f  
+ velocity: Vec2f  
+ scale:    Vec2f  
+ angle:    float
```

CCollision

```
+ radius:    float
```

CScore

```
+ score:     int
```

CShape

```
+ circle: sf::Circle
```

CLifespan

```
+ lifespan:  int  
+ remaining: int
```

CInput

```
+ up:      int  
+ down:    int  
+ left:    int  
+ right:   int  
+ shoot:   int
```

Entity

- Entity = Game Object
- Stores Component Tuple
- Tag = Entity 'type'
- Active = Alive or Dead
- ID = Integer identifier

Entity	
- m_components:	CTuple
- m_alive:	bool
- m_id:	int
- m_tag:	string
+ get<T>:	T&
+ add<T>:	void
+ has<T>:	bool
+ id:	int
+ isAlive:	bool
+ tag:	string&
+ destroy:	void

EntityManager

- Entity 'Factory' Class
- Delayed Entity Add()
 - No Iterator Invalidation
- Secondary map from tag->entity
 - Trade storage for convenience / run time
- Can do other book-keeping like mem mgmt

EntityManager	
- m_entities:	EntityVec
- m_entityMap:	EntityMap
- m_toAdd:	EntityVec
- m_totalEntities:	int
- init():	void
+ update():	void
+ addEntity(tag):	sp<Entity>
+ getEntities():	EntityVec&
+ getEntities(tag):	EntityVec&

Game

- Top-Level Game Object
- Holds all game data
- All game system functions
- All gameplay code

Game	
- m_window:	sf::Window
- m_entities:	EntityManager
- m_paused:	bool
- m_running:	bool
- init:	void
+ update:	void
// systems	
- sMovement:	void
- sUserInput:	void
- sEnemySpawner:	void
- sCollision:	void
- sRender:	void

Vec2

- 2D Math Structure
 - see previous lecture
- Templated Class
 - $T = (x,y)$ variable type

using namespace std; // use std::cout, std::endl

Vec2<T>	
+ x:	T
+ y:	T
+ operator ==:	bool
+ operator !=:	bool
+ operator +:	Vec2
+ operator -:	Vec2
+ operator *:	Vec2
+ operator /:	Vec2
...	
+ normalize:	void
+ length:	float

What can this engine do?

- Create game objects as Entity instances
- Add Component data to Entities
- Implement game play via Systems
 - Also, handle user input
- Pause game play / exit game
- Can init / load configuration from file

What are the limitations?

- Can only display one 'scene'
- Can NOT load texture / sounds assets
- Can NOT display textured animations
- Does not have any menu / interface

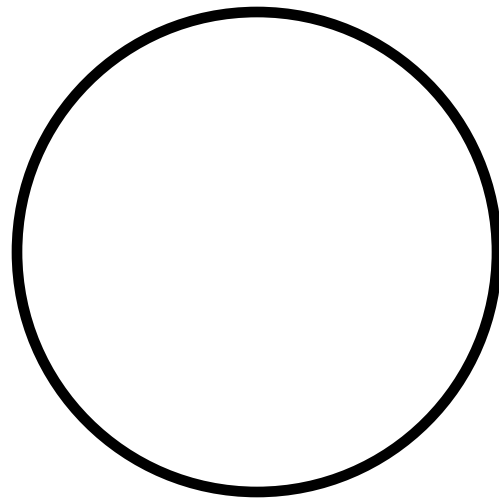
Game Scene

- A game can contain many different 'scenes' that have very different logic / controls
- Example: RPG Games
 - Menu / Text / Dialogue Scene
 - World-Map Scene
 - Combat / Battle Scene
- How can our architecture allow for different game states?



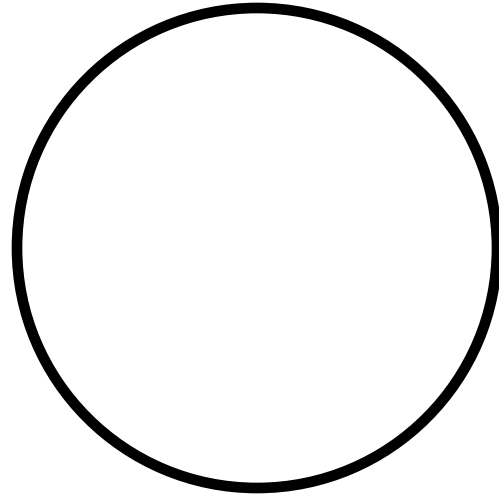
Circles

- Circles drawn on screen aren't true math circles
- Can't draw infinite points
- Decide on some number of points to draw
- Draw "enough" to look real on our screen



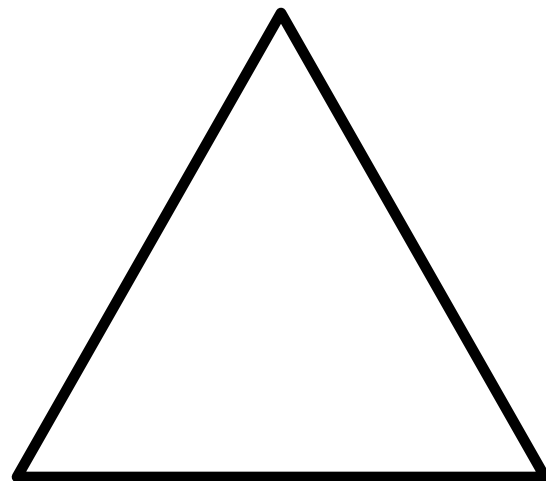
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- Example: `p=128`



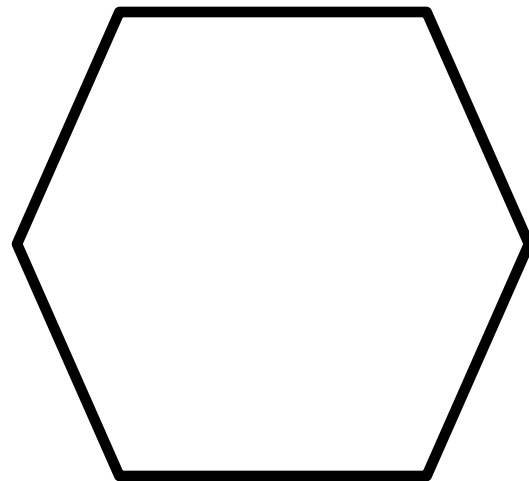
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(3);`



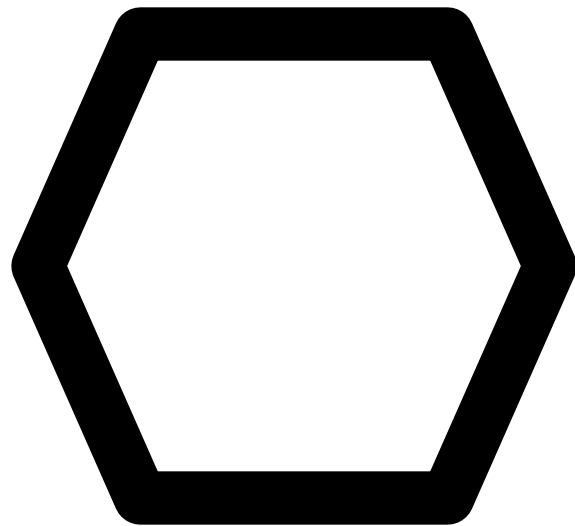
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(6);`



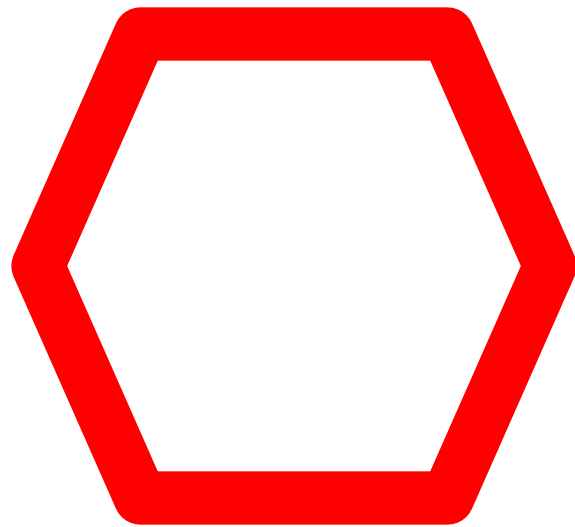
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(6);`
- `c.setOutlineThickness(20);`



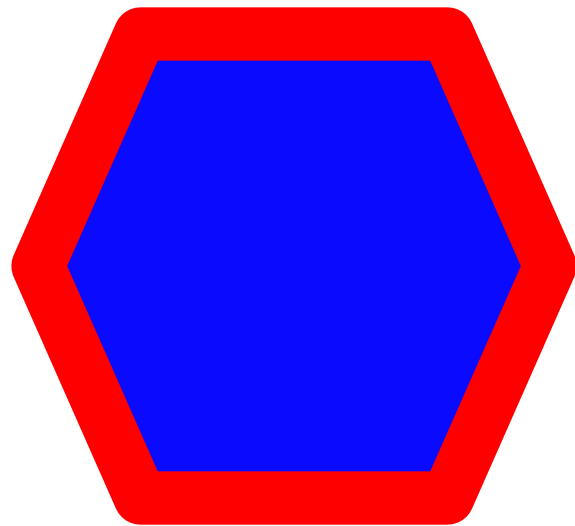
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(6);`
- `c.setOutlineThickness(20);`
- `c.setOutlineColor(red);`



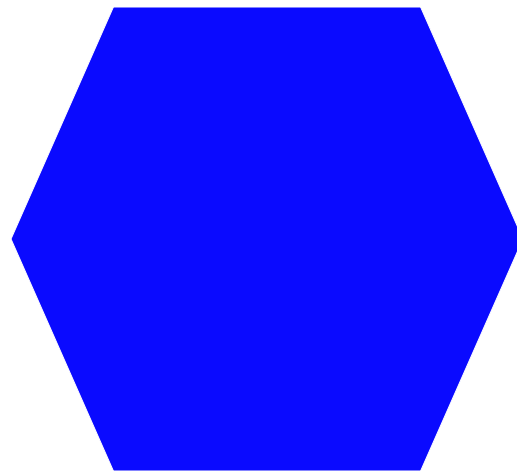
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(6);`
- `c.setOutlineThickness(20);`
- `c.setOutlineColor(red);`
- `c.setFillColor(blue);`



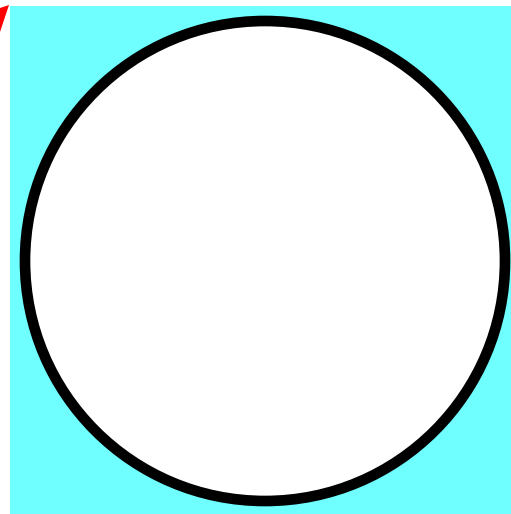
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setPointCount(6);`
- `c.setOutlineThickness(0);`
- `c.setOutlineColor(red);`
- `c.setFillColor(blue);`



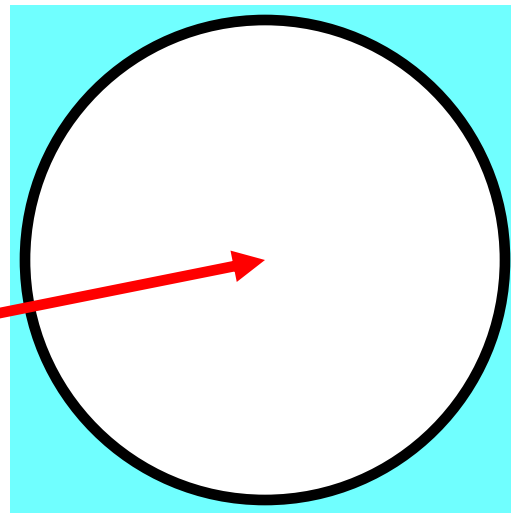
SFML Circles

- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setOrigin({0,0})`
 - `window.draw(c)` will draw the origin at `c`'s position

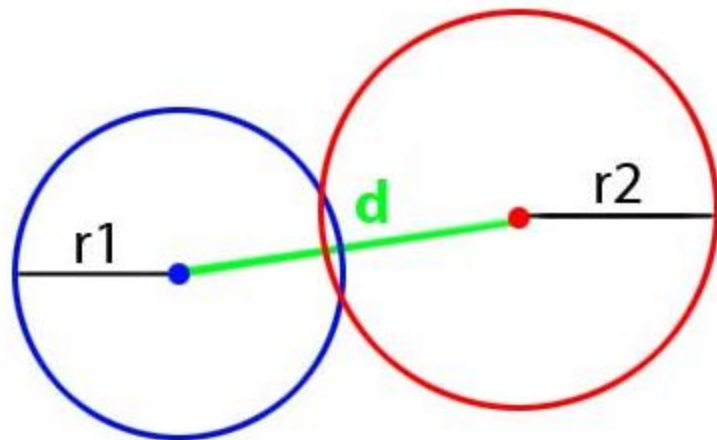
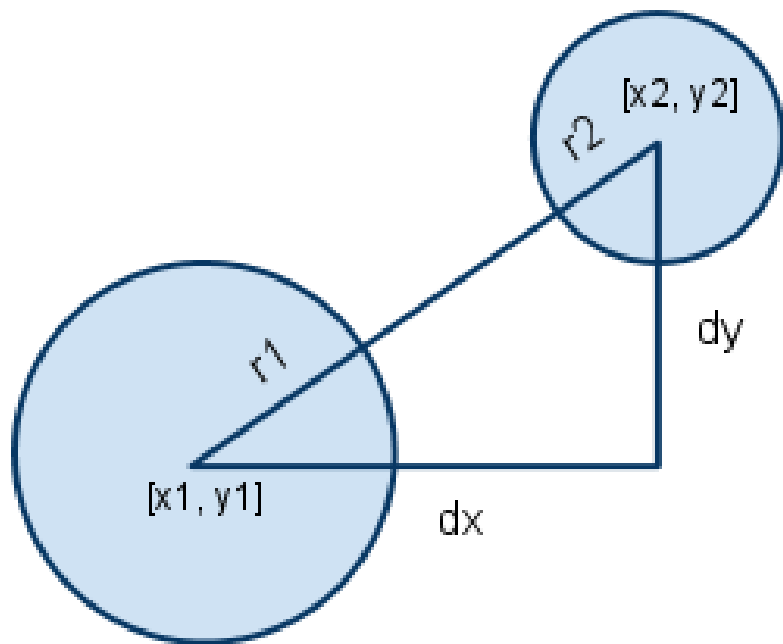


SFML Circles

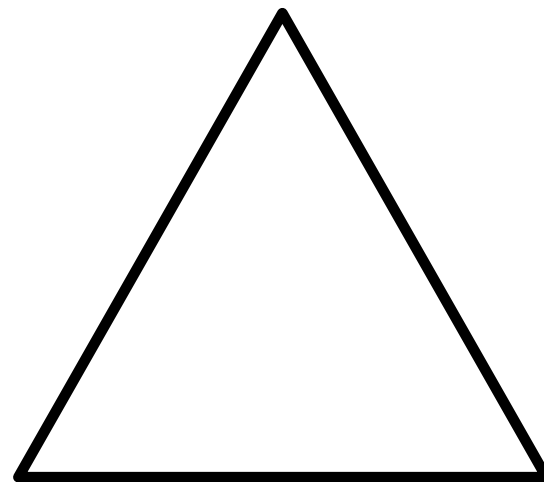
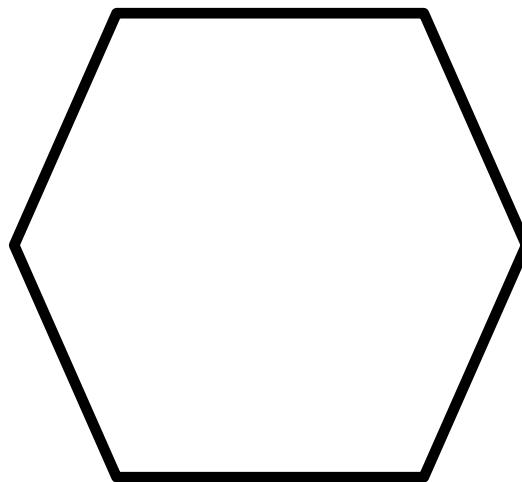
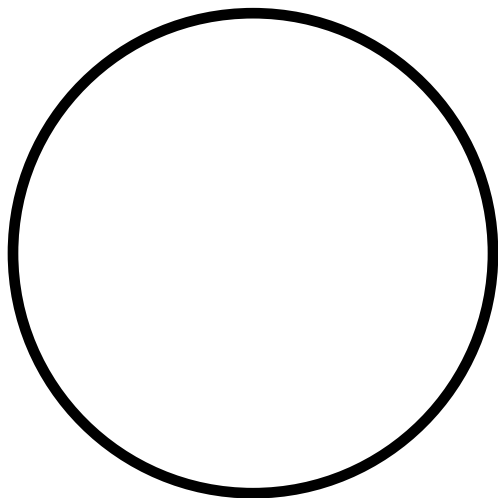
- `sf::CircleShape c(r, p);`
 - `r` = radius
 - `p` = point count
- `c.setOrigin({r,r})`
 - We want circle position to represent its center
 - Collision math easier



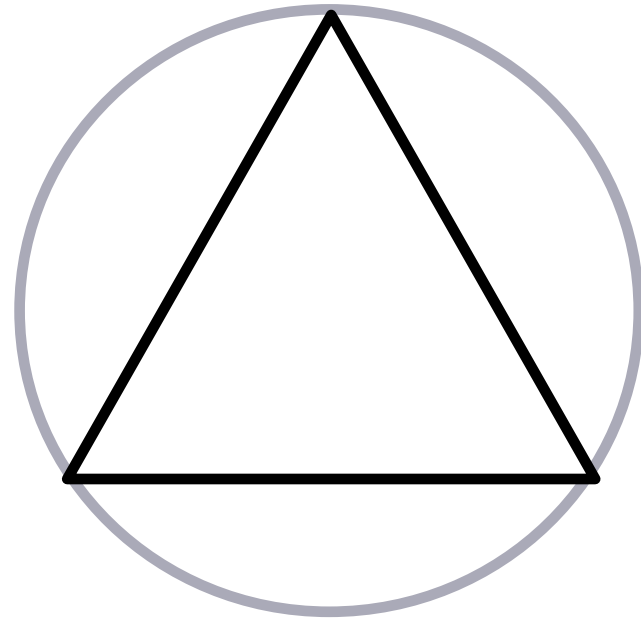
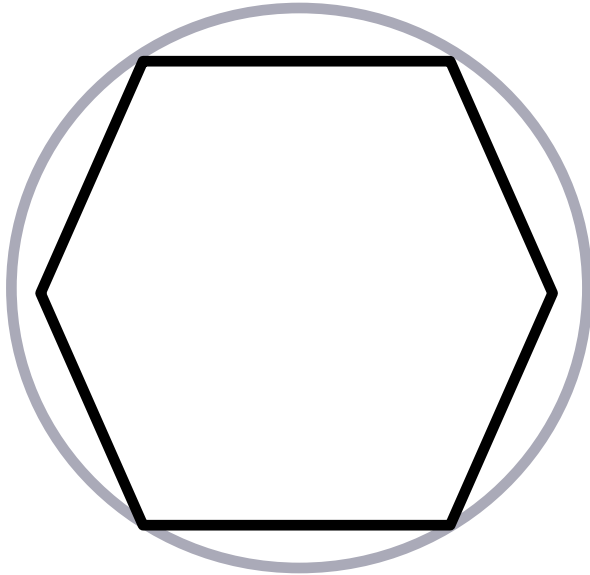
Circle Collisions



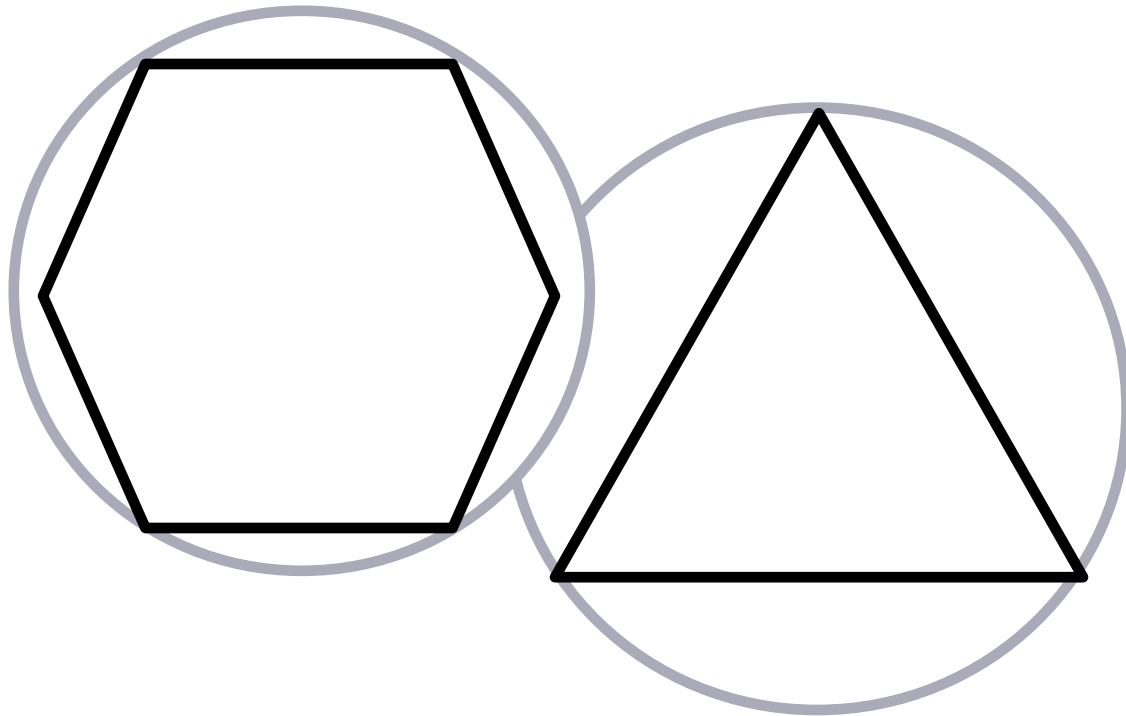
Shape Collisions



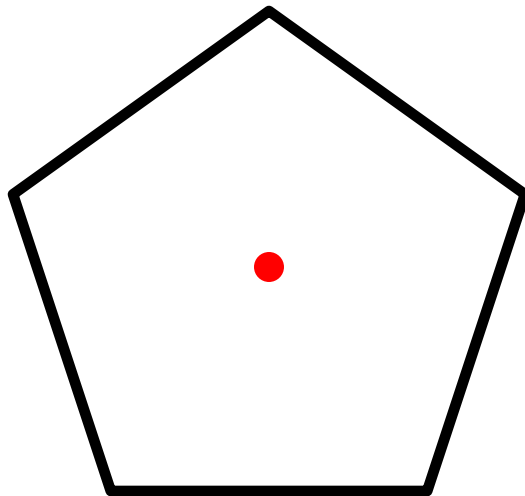
Shape Collisions



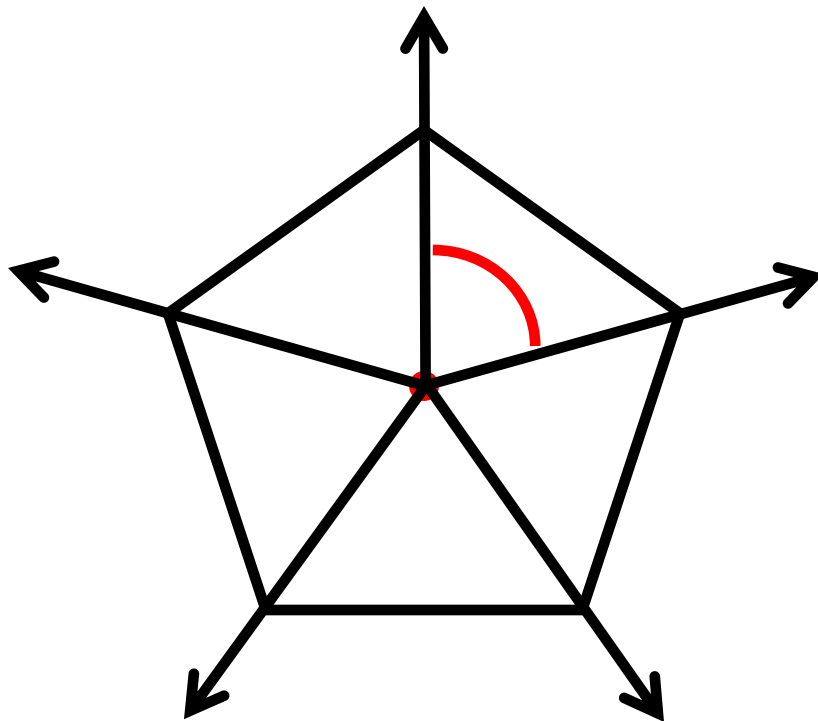
Shape Collisions



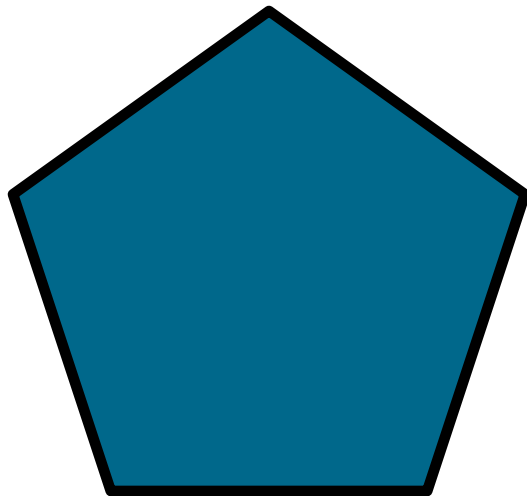
Small Enemy Spawning



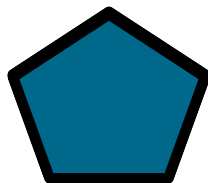
Small Enemy Spawning



Small Enemy Spawning



Small Enemy Spawning



Small Enemy Spawning